Setting up Ollama for the ERP assistant

Step-by-step for Windows with Laragon and an NVIDIA RTX 5060 Ti (16 GB)

How this guide was made. The steps follow the current Ollama documentation (docs.ollama.com) as read on 8 November 2026. They have not been run on your computer, so treat the expected outputs as what you should see, and compare. Model names change quickly: check ollama.com/library before you pull one.

1. What you are building

Ollama is a small program that runs language models on your own GPU and offers a local web address. Your Laravel app never talks to the GPU directly. It sends a normal HTTP request, and Ollama answers.

```
Browser -> Laravel / Livewire -> AssistantGateway (PHP)
        -> http://127.0.0.1:11434 -> Ollama -> RTX 5060 Ti
```

The assistant never gets database access. It can only call the PHP tools you give it (your existing summary services), and your code builds the chart data and validates everything. Ollama is only the part that chooses a tool and writes a sentence.

2. Before you start

Item	What you need	How to check
Windows	Windows 10 22H2 or newer (Home or Pro), or Windows 11	Win + R, type winver
NVIDIA driver	Documented minimum 551.61. An RTX 50-series card needs a recent driver, so install the latest from nvidia.com	Open PowerShell, run: nvidia-smi
Disk space	About 4 GB for Ollama, plus 5 to 20 GB per model	File Explorer, This PC
Laragon	Already running the ERP (PHP 8.4+)	php -v

Run `nvidia-smi` now. It should list the RTX 5060 Ti and about 16 GB of memory. If the command is not found, install or update the NVIDIA driver first and restart.

3. Install Ollama

- Download **OllamaSetup.exe** from ollama.com/download. It installs for your user and does not need Administrator rights.
- Run it. When it finishes, Ollama starts in the background (look for the llama icon in the system tray) and the `ollama` command works in any new terminal.
- Open a **new** PowerShell window and check:

```
ollama --version
```

The program lives in `%LOCALAPPDATA%\Programs\Ollama` and its logs in `%LOCALAPPDATA%\Ollama` (app.log and server.log). You will want those if anything goes wrong.

4. Optional: choose where models are stored

By default models go to `C:\Users\<you>\.ollama`. If drive C: is small, move them:

- Quit Ollama from the tray icon (right-click, Quit).
- Create the folder, for example `D:\ollama-models`.
- Set the variable, then start Ollama again from the Start menu:

```
[Environment]::SetEnvironmentVariable("OLLAMA_MODELS", "D:\ollama-models", "User")
```

5. Download a model

Your assistant needs a model that supports **tool calling** (so it can choose one of your summary tools) and can follow a JSON schema. With 16 GB, the goal is a model plus its working memory that stays under about 14 GB, so it runs 100% on the GPU.

Option	Pull command	Notes
Start here	ollama pull qwen3:8b	Small and fast. Qwen3 is the model used in Ollama's own tool-calling examples.
Step up	ollama pull qwen3:14b	Better judgement, still fits in 16 GB at normal settings.
Alternative	ollama pull gpt-oss:20b	Used in Ollama's structured-output examples. Close to the 16 GB limit, so keep the context length modest.

Check the exact tags and download sizes on each model's page at ollama.com/library (use the **tools** filter). Then talk to the model once to prove it works:

```
ollama run qwen3:8b
>>> Say hello in five words.
(type /bye to leave)
```

6. Check it is really using the GPU

While the model is loaded, open a second PowerShell window and run:

```
ollama ps
nvidia-smi
```

In `ollama ps`, the PROCESSOR column should read **100% GPU**. A split such as 48%/52% CPU/GPU means the model plus its context does not fit in 16 GB. The answers will be slow. Fix it by choosing a smaller model or lowering the context length (next section).

7. Settings that matter for the app

These are Windows **user** environment variables. To set them: quit Ollama from the tray, set them, then start Ollama again from the Start menu (a new terminal also picks them up).

Variable	Default	Suggested	Why
OLLAMA_CONTEXT_LENGTH	4096	8192	Room for tool descriptions plus results. Larger values use more VRAM.
OLLAMA_KEEP_ALIVE	5m	30m	Keeps the model loaded, so the first question after a quiet spell is not slow.
OLLAMA_MAX_LOADED_MODELS	3 per GPU	1	Stops a second model from squeezing the first out of VRAM.
OLLAMA_NUM_PARALLEL	1	1	Parallel requests multiply memory use. Keep 1 and queue requests in Laravel.
OLLAMA_HOST	127.0.0.1:11434	leave alone	Only change it if the app runs on a different computer (section 11).

```
[Environment]::SetEnvironmentVariable("OLLAMA_CONTEXT_LENGTH", "8192", "User")
[Environment]::SetEnvironmentVariable("OLLAMA_KEEP_ALIVE", "30m", "User")
[Environment]::SetEnvironmentVariable("OLLAMA_MAX_LOADED_MODELS", "1", "User")
```

After restarting Ollama and loading the model, run `ollama ps` again and confirm it is still 100% GPU.

8. Test the web API from PowerShell

Ollama answers at `http://localhost:11434`. Use `"stream": false` so you get one complete answer. **Test A: a plain answer.**

```
$body = '{"model":"qwen3:8b","stream":false,"messages":
  [{"role":"user","content":"Say hello in five words."}]}'
Invoke-RestMethod -Uri http://localhost:11434/api/chat -Method Post `
  -ContentType "application/json" -Body $body
```

The reply has a `message.content` field with the text. The first call is slower while the model loads.

Test B: tool calling. This is what the chart assistant will do. We describe one tool and see whether the model asks for it.

```
$body = @'
{
  "model": "qwen3:8b",
  "stream": false,
  "messages": [{"role": "user", "content": "How old are our assets?"}],
  "tools": [{
    "type": "function",
    "function": {
      "name": "age_buckets",
      "description": "Count of assets by age band",
      "parameters": {"type": "object", "properties": {}, "required": []}
    }
  }]
}
'@
Invoke-RestMethod http://localhost:11434/api/chat -Method Post `
  -ContentType "application/json" -Body $body
```

Success: `message.tool_calls` contains a call whose `function.name` is `age_buckets`. Your PHP then runs the real tool and sends the result back as a message with role `tool`, a `tool_name` and the `content`. The model then answers in words.

Test C: structured output. A JSON schema in `format` forces the reply into a shape your code can check.

```
$body = @'
{
  "model": "qwen3:8b",
  "stream": false,
  "messages": [{"role": "user", "content": "Pick a chart for asset age by region."}],
  "options": {"temperature": 0},
  "format": {
    "type": "object",
    "properties": {
      "chart_type": {"type": "string", "enum": ["bar", "line", "doughnut"]},
      "title": {"type": "string"}
    },
    "required": ["chart_type", "title"]
  }
}
'@
Invoke-RestMethod http://localhost:11434/api/chat -Method Post `
  -ContentType "application/json" -Body $body
```

Ollama's guidance for reliable structured output: set the temperature to 0, and also describe the schema in the prompt. Qwen3 can spend time on internal reasoning before it answers. If replies feel slow, check the Ollama docs for the *think* option for your model.

9. Test it from Laravel

Following your project's convention, new settings go in `config/gwl.php` and are read with `config('gwl.your_key')`. Add these to `.env`:

```
GWL_ASSISTANT_ENABLED=false
GWL_ASSISTANT_PROVIDER=ollama
GWL_OLLAMA_URL=http://127.0.0.1:11434
GWL_OLLAMA_MODEL=qwen3:8b
GWL_OLLAMA_TIMEOUT=120
```

The keys are read in `config/gwl.php` (for example `'ollama_url' => env('GWL_OLLAMA_URL', 'http://127.0.0.1:11434')`). The flag stays **false** until the feature is built. Then open `php artisan tinker` and run:

```
use Illuminate\Support\Facades\Http;

$r = Http::timeout(120)->post('http://127.0.0.1:11434/api/chat', [
    'model'    => 'qwen3:8b',
    'stream'   => false,
    'messages' => [['role' => 'user', 'content' => 'Say hello in five words.']],
    'options'  => ['temperature' => 0],
]);

$r->status();                  // 200
$r->json('message.content');   // the answer
```

Two things to remember when this goes into the real feature:

- **Set a generous HTTP timeout.** The first request after a quiet period loads the model into the GPU and can take many seconds. Laravel's default is shorter.
- **Do not hold up a web request.** Run slow work (and later, any heavy extraction) on the queue, or call it from a Livewire action that shows a spinner. One GPU serves one request at a time, so several users would wait in line.

10. Quick health checks you can reuse

Check	Command	Healthy result
Server is up	Invoke-RestMethod http://localhost:11434/api/tags	A list of your downloaded models
Model is loaded	ollama ps	Model name, 100% GPU
GPU memory	nvidia-smi	Memory in use close to model size, under 16 GB
Port is listening	netstat -ano | findstr 11434	A LISTENING line

11. If the app moves to a server later

By default Ollama listens on `127.0.0.1`, so only programs on the **same computer** can reach it. That is perfect for development. If the ERP is later hosted elsewhere and your GPU PC stays where it is, the app will not be able to reach it. You would then need a GPU machine that stays switched on, with Ollama exposed to the app server only.

- Set `OLLAMA_HOST` to `0.0.0.0:11434` on the GPU machine and restart Ollama.
- Ollama has **no login of its own**. Never expose the port to the internet. Allow only the app server's address:

```
New-NetFirewallRule -DisplayName "Ollama (ERP server only)" -Direction Inbound `
  -Protocol TCP -LocalPort 11434 -RemoteAddress <APP_SERVER_IP> -Action Allow
```

- The tray app only runs while someone is signed in. For an always-on machine, Ollama's docs describe the standalone `ollama-windows-amd64.zip` run with `ollama serve` under a service manager such as NSSM.

12. Troubleshooting

Problem	Likely cause and fix
ollama is not recognised	Open a **new** terminal after installing. If it still fails, sign out and back in so PATH refreshes.
Very slow answers	Run ollama ps. If it is not 100% GPU, use a smaller model or lower OLLAMA_CONTEXT_LENGTH. Update the NVIDIA driver.
First request very slow, later ones fast	Normal: the model is loading. Raise OLLAMA_KEEP_ALIVE (section 7).
Laravel: cURL error 28 (timeout)	Increase Http::timeout(). Check the model is not still loading, and that no other request holds the GPU.
Laravel: connection refused	Ollama is not running (start it from the Start menu), or the URL or port is wrong, or you pointed at a different computer.
model not found	The name in GWL_OLLAMA_MODEL does not match. Run: ollama list, and copy the exact name and tag.
Model ignores the tools	It may not support tool calling. Use a model with the tools tag. Lower the temperature to 0.
Odd or invalid JSON	Use the format schema (Test C) and validate in PHP anyway. Never trust the model's output unchecked.
Anything else	Read %LOCALAPPDATA%\Ollama\server.log and app.log.

13. Safe-use checklist for this ERP

- Keep Ollama on 127.0.0.1 unless you have a reason and a firewall rule.
- The model sees only what your PHP tools return. Never give it raw table access or a database login.
- Keep personal data (customer names and phones, staff details, anonymous incident reports, signatures, network passwords) out of every tool result and every prompt.
- Do not write prompts or model replies to logs if they could contain personal data.
- Validate everything the model returns (tool names, chart types, ids) in PHP before using it.
- Run the model as a read-only helper first. Any save still goes through the normal screens and services, so existing validation and audit logging apply.

14. Done when

- `ollama --version` works and nvidia-smi shows the RTX 5060 Ti.
- `ollama ps` shows your model at 100% GPU with the settings from section 7.
- Test B returns a tool call and Test C returns valid JSON.
- The tinker snippet returns HTTP 200 with a sensible answer.

Next step: build the AssistantGateway (Ollama provider plus a fake for tests) and the asset serial lookup, using the values from your .env.